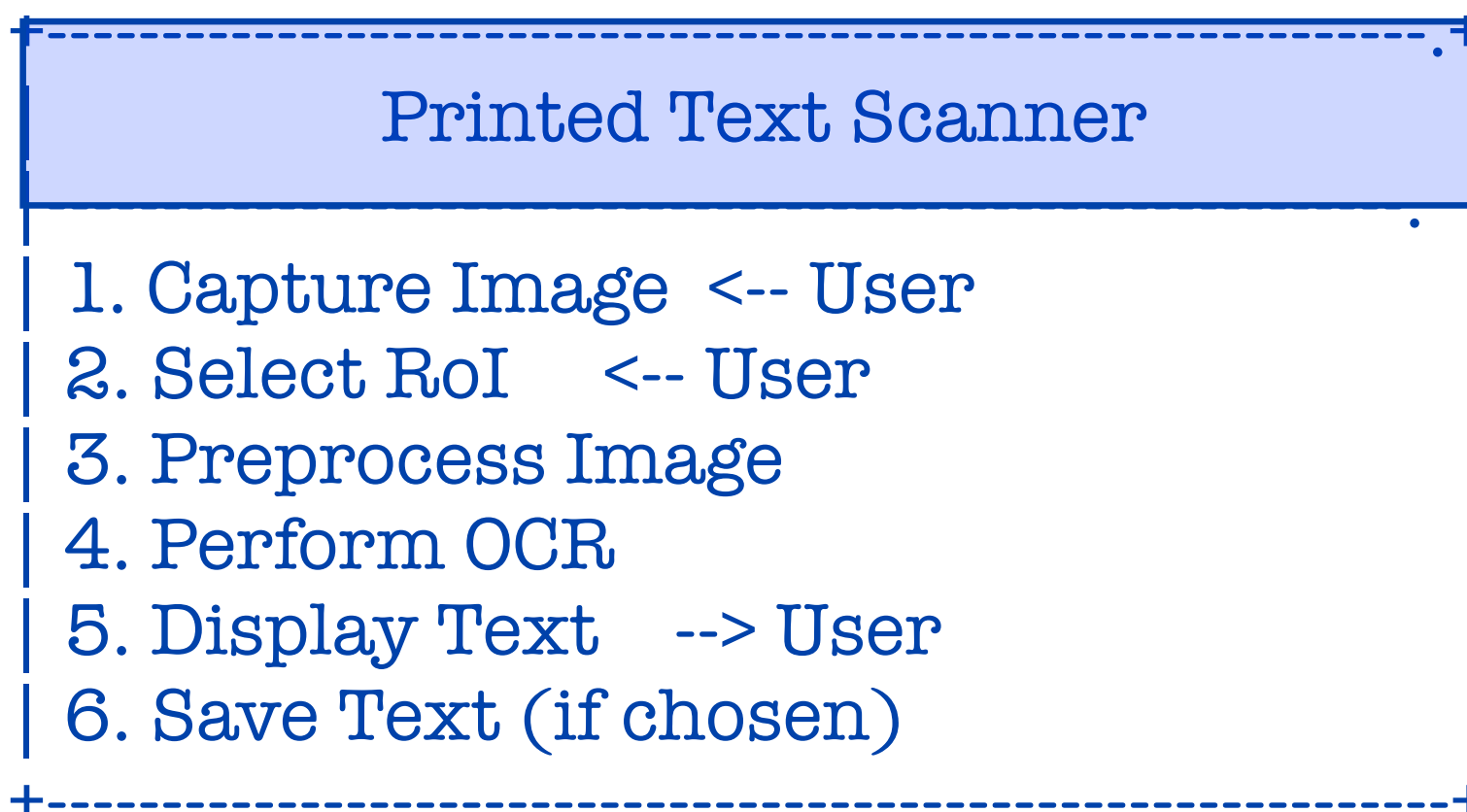


Printed Text Scanner Project

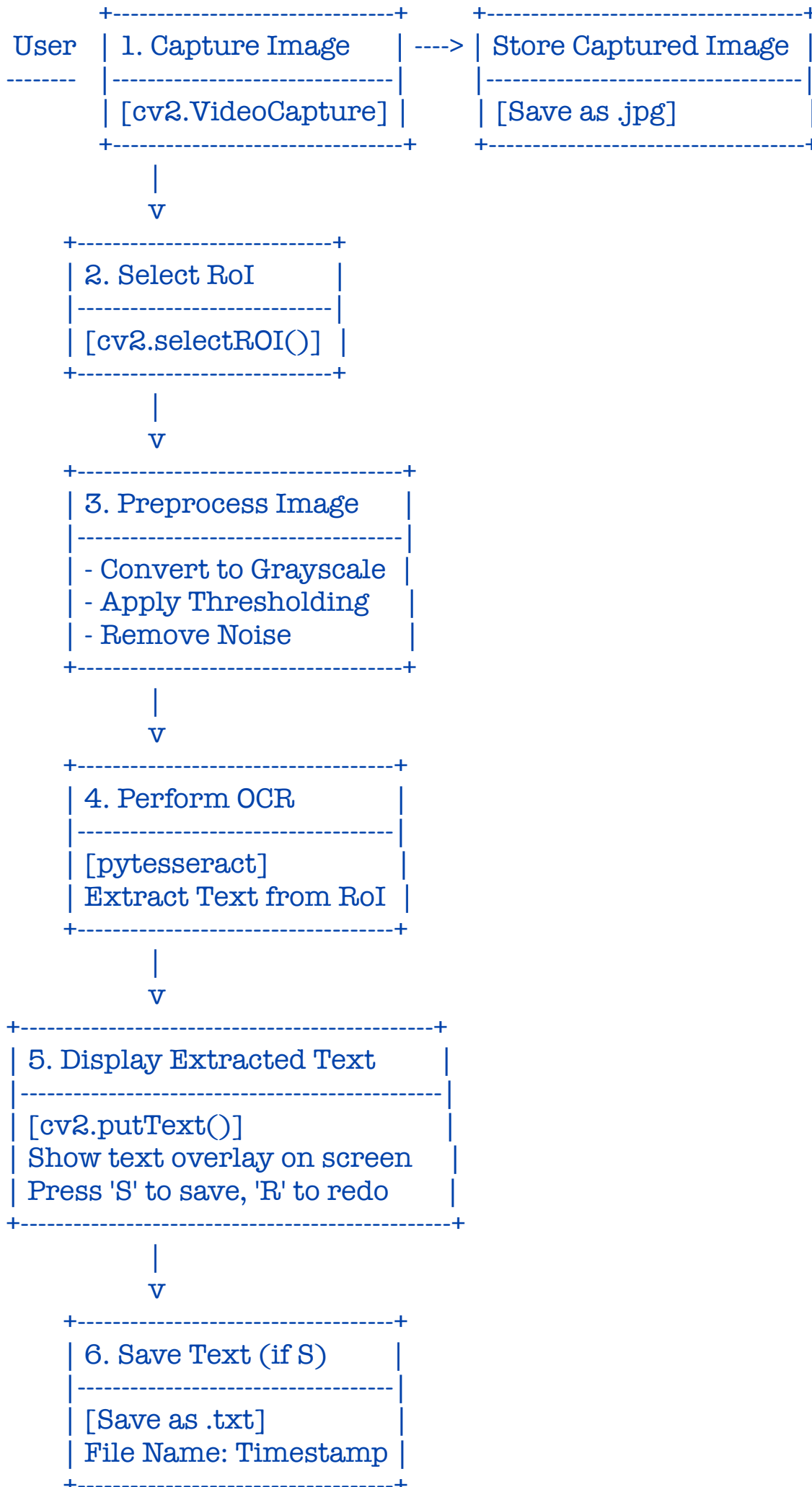
One of the Applications of OpenCV and Tesseract

Data Flow Diagram (DFD)

Level 0 DFD: Context Diagram



Level 1 DFD: Breaking Down the Process



1 Camera Handling & Image Capture

◆ Function: `capture_image()`

◆ What the function does:

- ✓ OpenCV's `cv2.VideoCapture()` – How to access the camera.
- ✓ Frame Display & User Interaction – Showing live instructions.
- ✓ Handling Camera Jamming – Implementing retries if the camera fails.
- ✓ Releasing Camera Resources – Using `cap.release()` & `cv2.destroyAllWindows()`.

2 Selecting Region of Interest (Roi)

◆ Function: `select_roi()`

◆ What the function does:

✓ Using `cv2.selectROI()` – Allowing users to drag and select a portion of the image.

✓ Understanding Coordinates (x, y, width, height) – Extracting a specific area.

✓ Validating Roi Selection – Ensuring the user doesn't select an empty box.

3 Image Preprocessing for OCR

◆ Function: `preprocess_image()`

◆ What the function does:

✓ Grayscale Conversion – `cv2.cvtColor()` to convert images for better text recognition.

✓ Noise Reduction – Using `cv2.GaussianBlur()`.

✓ Thresholding (`cv2.threshold()`) – Making text more readable for OCR.

✓ Why Preprocessing is Important for OCR Accuracy

4 Optical Character Recognition (OCR)

◆ Function: `extract_text_from_image()`

◆ What the function does:

✓ Using pytesseract for OCR – Extracting text from images.

✓ Text Processing & Cleaning – Removing unwanted characters, line breaks.

✓ OCR Modes (`--psm 6`) – Understanding different modes for structured/unstructured text.

✓ Performance & Accuracy Tuning

5 Saving & Displaying Extracted Text

◆ Functions:

`extract_text_and_display()`,
`save_text()`,
`cleanup_and_exit()`

◆ What the functions do:

- ✓ Displaying Text on an Image (`cv2.putText()`)
- ✓ Creating & Managing Unique Files (`time.strftime()`)
- ✓ Saving Text as .txt Files (`open()` with utf-8 encoding)
- ✓ Clearing Temporary Images to Optimize Storage